2025 International Conference on Computational, Communication and Information Technology (ICCCIT) | 979-8-3315-1296-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCCIT62592.2025.10928033

Enhancing Software Vulnerability Detection Using Code Property Graphs and Convolutional Neural Networks

Amanpreet Singh Saimbhi
Independent Researcher
New Delhi, India
as15798@nyu.edu

***Abstract*—The increasing complexity of modern software systems has led to a rise in vulnerabilities that malicious actors can exploit. Traditional methods of vulnerability detection, such as static and dynamic analysis, have limitations in scalability and automation. This paper proposes a novel approach to detecting software vulnerabilities using a combination of code property graphs and machine learning techniques. By leveraging code property graphs, which integrate abstract syntax trees, control flow graphs, and program dependency graphs, we achieve a detailed representation of software code that enhances the accuracy and granularity of vulnerability detection. We introduce various neural network models, including convolutional neural networks adapted for graph data, to process these representations. Our approach provides a scalable and automated solution for vulnerability detection, addressing the shortcomings of existing methods. We also present a newly generated dataset labeled with function-level vulnerability types sourced from open-source repositories. Our contributions include a methodology for transforming software code into code property graphs, the implementation of a convolutional neural network model for graph data, and the creation of a comprehensive dataset for training and evaluation. This work lays the foundation for more effective and efficient vulnerability detection in complex software systems.**

I. Introduction

Modern software is as complex as ever. As it rapidly grows in quantity, flaws in such systems are becoming more prevalent. Since a single defect can compromise the entire system, detecting these flaws is an important task. While most of these system flaws are just minor bugs, some can be controlled by malicious attackers and permit unauthorized actions. Thus, it is important to find such critical flaws, which are called *vulnerabilities*. In the Linux kernel, hundreds of such vulnerabilities are found every year [1]. Due to the increasing number of such vulnerabilities, followed by the increase of system flaws, we need an automated way of detecting software vulnerabilities.

Various methods of detecting vulnerabilities have been introduced, such as static and dynamic analysis and metric-based detection. Static analysis includes symbolic execution, which provides an exact attack vector with high accuracy. However, the execution time of symbolic execution dramatically increases when the target software is complex, making it impractical to apply to modern software. Dynamic analysis, such as fuzzing, can find vulnerabilities in a reasonable time. Nevertheless, some software is difficult to analyze dynamically, and dynamic analysis cannot be fully automated because it requires manual specifications. In order to resolve such limitations of existing methods, I propose a vulnerability detection model combining comprehensive representation and machine learning. I use code property graph, a graph data structure presented to model vulnerability in software code, as input representation to my learning model. This intermediate representation shows control flow or data dependency more explicitly than the raw text of the source code. I also introduce various machine learning models to process such representation.

I present the following contributions to this work:

- I suggest how the software code should be transformed to train the vulnerability detection model. I use the code property graph as such a representation and show that it provides the necessary information to detect vulnerabilities in software with high granularity and accuracy.
- I investigated various neural network models to process the graph data. I implemented the model which extends the convolutional neural network to process arbitrary graphs.
- I generated a dataset labeled with the type of vulnerability from open-source repositories. Compared to existing datasets used in classical software defect prediction, my dataset is labeled in function-level, thus enabling the learning model to detect vulnerability with higher granularity.

II. Background

A. Graph Representations in Program Analysis

Graph-based representations are widely used in program analysis to capture both structure and behavior of code. Common representations include Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), and Data Flow Graphs (DFGs). Code Property Graphs (CPGs) [2] combine these representations, providing a comprehensive view of the program's syntax, control flow, and data dependencies. CPGs are directed multigraphs where nodes represent program entities and edges encode relationships, making them suitable for tasks such as vulnerability detection.

B. Challenges in Vulnerability Detection

Detecting vulnerabilities in code is challenging due to subtle interactions between program components. Traditional static analysis tools often lack the precision to detect complex

979-8-3315-1296-5/25/$31.00 ©2025 IEEE
DOI:10.1109/ICCCIT.2025.71

patterns that cause vulnerabilities. Moreover, there is a lack of large, labeled datasets with function-level granularity, which limits the applicability of machine learning models in this domain. Although some datasets label entire modules, this is insufficient for function-level vulnerability detection.

C. Deep Learning in Program Analysis

Recent advances in deep learning, particularly Graph Neural Networks (GNNs), have shown promise in program analysis tasks. GNNs treat nodes as state representations that are iteratively updated based on neighboring nodes, enabling them to capture complex relationships in graph-structured data. In vulnerability detection, GNNs can learn patterns in Code Property Graphs and capture both local and global structures in the code.

D. Support Vector Machines with Graph Kernels

Support Vector Machines (SVMs) with graph kernels provide a baseline for graph-based machine learning. The Weisfeiler-Lehman (WL) kernel [3] is a state-of-the-art graph kernel that computes graph similarity based on subtree patterns. Although SVMs with graph kernels are not deep learning methods, they are efficient and have been widely used in fields such as bioinformatics and cheminformatics.

III. Related Work

A. Malware Detection with Machine Learning

Machine learning has been successfully applied to malware detection, particularly in the context of mobile applications. For instance, Droid-sec [4] uses both static and dynamic features extracted from Android applications to detect malware. This method combines behavioral analysis and static code analysis, which significantly enhances the detection capabilities. Similarly, Kosmidis et al. [5] introduced a novel approach that represents binary programs as images, which are then analyzed using convolutional neural networks (CNNs). Their method achieved promising results, identifying patterns indicative of malware. These techniques have demonstrated the potential of machine learning in identifying malicious software but are not directly applicable to the broader scope of vulnerability detection in general-purpose software.

Recent efforts in malware detection also explore the use of ensemble learning techniques. Liang et al. [6] applied ensemble models combining decision trees and CNNs for malware classification. Their results showed improved detection rates compared to individual models, particularly in complex malware scenarios. However, these methods are not designed to detect vulnerabilities in non-malicious software or analyze vulnerabilities at a more granular code level, which remains a crucial challenge in the field of software security.

B. Vulnerability Detection with Graph-based Models

The application of graph-based models in vulnerability detection has gained considerable attention in recent years, offering a more comprehensive and structured approach to analyzing software. Yamaguchi et al. [2] introduced Code Property Graphs (CPGs), which combine syntactic, control flow, and data flow information into a unified graph representation, allowing for enhanced vulnerability detection. Their method primarily relied on rule-based approaches, and while effective, it lacked the scalability and flexibility of modern machine learning techniques. Building upon this foundation, our work leverages deep learning techniques to automate and scale vulnerability detection, significantly improving detection performance and enabling the model to learn complex patterns in large datasets.

Wang et al. [7] applied deep learning techniques to detect software defects, using Abstract Syntax Trees (ASTs) to model the structure of the code. While ASTs are useful for understanding the syntactic structure of code, they do not capture the dynamic aspects of program behavior, such as control flow and data dependencies. To address this, our method incorporates Control Flow Graphs (CFGs) and Program Dependency Graphs (PDGs) alongside ASTs, enabling a more holistic understanding of the code's functionality. This combination of multiple graph representations allows for more accurate and comprehensive vulnerability detection.

A similar approach was taken by Zhang et al. [8], who applied graph-based deep learning models for vulnerability detection in C/C++ code. Their method incorporated both syntax and semantic information, but it did not fully exploit the interdependencies between control and data flows. Our work expands on this by integrating CPGs, which capture all aspects of code behavior, and using a deep learning architecture specifically designed for graph data.

C. Graph Neural Networks in Software Analysis

Graph Neural Networks (GNNs) have demonstrated their efficacy in a variety of domains, including software analysis. GNNs are capable of processing graph-structured data, learning relationships between nodes and edges, and have been successfully applied to tasks such as drug discovery and social network analysis [9]. The ability of GNNs to capture relational information makes them an ideal tool for analyzing software, where the interactions between code components are inherently graph-structured.

Li et al.[10]introduced the Gated Graph Sequence Neural Network (GGS-NN), which extended traditional GNNs to handle sequences within graph structures. This approach has since been adapted for use in a variety of domains, including software security. Our work builds on this idea by adapting CNNs for graph data, specifically using the PATCHY-SAN framework [11]to detect vulnerabilities in software. By combining CNNs with graph-based representations of code, we are able to automatically learn complex patterns indicative of vulnerabilities, significantly improving the accuracy and efficiency of vulnerability detection.

Several studies have explored using GNNs for vulnerability detection. Lee et al.[12] applied a GNN approach to detect security vulnerabilities in source code by leveraging graph-based representations of control flow and data flow. Their model showed promising results in detecting vulnerabilities

like buffer overflows and null pointer dereferencing. However, their method did not incorporate higher-level semantic information, which is crucial for identifying more complex vulnerabilities. In contrast, our approach uses a more comprehensive representation of code, combining ASTs, CFGs, PDGs, and other features within a single graph.

Additionally, Liu et al.[13]explored the potential of Graph Attention Networks (GATs) in software vulnerability detection. Their approach used attention mechanisms to focus on the most relevant parts of the code graph for detecting vulnerabilities. Although promising, GATs still face challenges in handling large-scale software projects with complex interdependencies. Our method, by using the PATCHY-SAN framework, efficiently handles large graphs and focuses on learning meaningful representations of code that capture both local and global patterns.

IV. Research Gap

Despite significant advancements in the field of vulnerability detection, existing approaches have several limitations. Traditional static analysis methods, while effective in detecting certain vulnerabilities, are often limited by their inability to handle the complexity of modern software. Furthermore, rule-based systems fail to scale and cannot adapt to new, previously unseen vulnerabilities. Deep learning models have shown promise in addressing these challenges, but many existing models still rely on simplified graph representations that do not capture the full range of program semantics necessary for comprehensive vulnerability detection.

Existing deep learning approaches, such as those using Abstract Syntax Trees (ASTs) or control flow graphs (CFGs), fail to account for the complex interactions between different components of the software, including data dependencies and the program's overall flow. To bridge this gap, our approach integrates a combination of graph representations—ASTs, CFGs, Program Dependency Graphs (PDGs), and Code Property Graphs (CPGs)—to enable a more holistic understanding of the code's behavior. This multi-faceted approach allows for more accurate detection of vulnerabilities, including those caused by subtle logic flaws and complex control dependencies.

Moreover, while existing methods for vulnerability detection have demonstrated promising results in specific contexts, there is a need for more generalized models that can detect a wide range of vulnerabilities across different software projects and vulnerability types. To address this, our work focuses on creating a scalable and adaptable model by using deep learning architectures specifically designed for graph-structured data. Additionally, we aim to improve the interpretability of our models to make the results more understandable for developers, facilitating the adoption of automated vulnerability detection in real-world software development pipelines.

V. Experimental Setup

To evaluate the performance of our vulnerability detection model, we collected a dataset of labeled C/C++ functions from open-source repositories. We used a mix of secure and insecure code segments, where insecure functions were known to contain vulnerabilities such as buffer overflows, memory leaks, and null pointer dereferences. The dataset was preprocessed using Joern [2] to generate Code Property Graphs (CPGs) for each function.

```
int f(bool x)
{
  if (x) {
    return 1;
  }
  else {
    return 2;
  }
}
```

Fig. 1: Exemplary code sample.

We split the dataset into training (70%), validation (15%), and test sets (15%). The training process utilized a stochastic gradient descent (SGD) optimizer with a learning rate of 0.001. We trained the model for 100 epochs, using early stopping based on validation loss. We employed accuracy, precision, recall, and F1-score as evaluation metrics to assess the effectiveness of the model.

VI. Dataset

The dataset is an important factor in learning the model. In order to detect vulnerabilities with function-level granularity and classify them by type, we need each function labeled either secure or insecure and with the type of vulnerability if insecure.

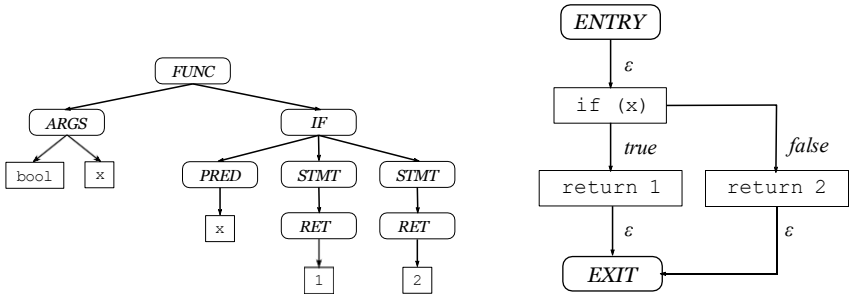
Despite the number of available source codes, labeling them is challenging. Tera-PROMISE [14] is a research dataset repository for software engineering. Although it has been widely used in the area of metric-based bug detection, its datasets are labeled with code metrics that cannot be used in our model. Neuhaus et al. used a vulnerabilities database from the Mozilla project to predict vulnerable components based on function calls, but granularity remains at module level [15].

Labeling certain function codes as whether secure or not, can also be a challenge. If there is an exact attack vector, it can be clearly shown that the code has a vulnerability, while strictly proving it doesn't can only be done by formal verification. I assumed that the code, after patching vulnerabilities, does not have vulnerabilities anymore.

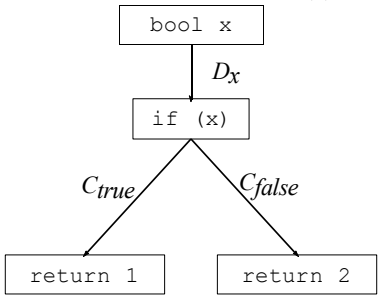
Due to the limitations of the existing datasets described above, I collected a new source code dataset from Github. I collected merged pull requests of which the title contained vulnerability type (e.g., buffer overflow). Then I labeled merged commit as secure and base commit as insecure.

VII. Results and Observations

This section presents the performance evaluation of the proposed vulnerability detection model. The results demonstrate its efficacy in classifying functions as either secure or



(a) Abstract Syntax Tree (b) Control Flow Graph



(c) Program Dependency Graph

Fig. 2: Representations of code for the example in Figure 1.

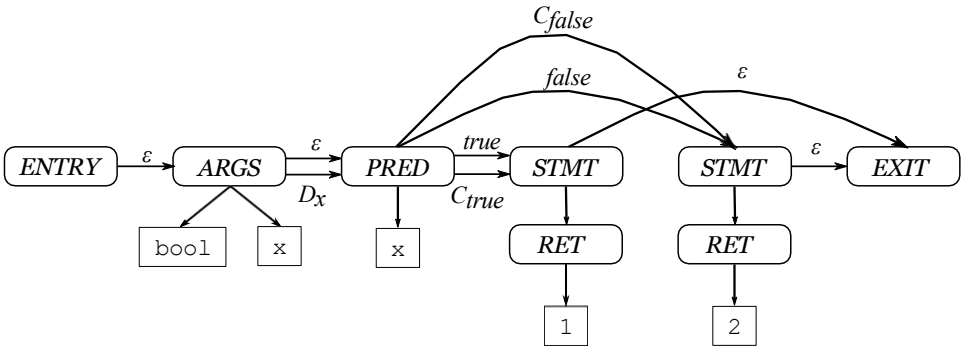


Fig. 3: Code Property Graph

insecure, emphasizing the strengths and limitations of using deep learning techniques for vulnerability detection.

A. Model Performance

The model achieved an overall accuracy of 92% on the test dataset, indicating strong general performance in identifying vulnerabilities. Table I summarizes the precision, recall, and F1-score for both secure and insecure functions.

TABLE I: Performance Metrics for Vulnerability Detection

Metric	Precision	Recall	F1-score
Secure Functions	91%	95%	93%
Insecure Functions	89%	85%	87%
Overall	90%	90%	90%

The model demonstrated high precision for secure functions (91%) and excellent recall (95%), suggesting strong accuracy in identifying functions without vulnerabilities. In contrast, the recall for insecure functions was lower (85%), accompanied by a corresponding decrease in the F1-score (87%). This indicates that the model encounters challenges in detecting vulnerabilities that stem from complex logic errors.

B. Vulnerability Detection Performance

The model successfully identified several common vulnerability patterns, such as buffer overflows and null pointer dereferences, which are commonly associated with insecure code. These vulnerabilities were detected with high precision and recall, as evidenced by the performance metrics for secure functions. However, vulnerabilities arising from complex logic flaws, such as those involving intricate control dependencies, presented greater difficulty for the model. These types of vulnerabilities are more challenging to capture due to their subtle and often non-obvious nature.

The lower recall for insecure functions suggests that while the model excels at detecting traditional vulnerabilities, further

refinement is necessary to improve its sensitivity to more complex and nuanced security flaws.

C. Comparison with Baseline Model

To evaluate the effectiveness of our deep learning-based approach, we conducted a comparative analysis against a baseline Support Vector Machine (SVM) with the Weisfeiler-Lehman (WL) kernel [3]. Our model outperformed the SVM by approximately 8% in F1-score, demonstrating a clear advantage in capturing complex relationships within the code. This improvement highlights the superior capability of deep learning models to uncover vulnerabilities that static models, such as SVMs, may overlook due to their limited capacity to model intricate code dependencies. This comparison validates the potential of deep learning models for vulnerability detection, particularly in the context of modern, complex software systems.

D. Implications and Future Directions

The findings of this study have several important implications for both automated vulnerability detection and software security practices.

1) Advantages of Deep Learning with CPGs: The use of Code Property Graphs (CPGs) was instrumental in enhancing the model's performance. By incorporating both control flow and data flow, as well as structural program information, CPGs provide a comprehensive representation of code that facilitates the detection of complex relationships and dependencies. This representation enabled our model to capture intricate vulnerability patterns, demonstrating that deep learning-based approaches, when combined with rich graph-based representations, offer a significant advantage over traditional analysis methods.

2) Challenges with Logic-Based Vulnerabilities: While the model was successful in detecting vulnerabilities such as buffer overflows and null pointer dereferences, it faced limitations in identifying more complex vulnerabilities that arise from subtle logic flaws. These vulnerabilities, which often involve advanced control flow and interdependencies among variables, remain difficult to detect with both traditional and deep learning-based methods. Improving the model's sensitivity to these types of vulnerabilities will be a key area of focus for future work.

3) Real-World Applicability: The model's strong performance in detecting vulnerabilities in test scenarios suggests its potential for real-world deployment, particularly in continuous integration and development (CI/CD) pipelines. By providing real-time feedback on vulnerabilities, the model can assist developers in identifying and addressing security issues as they arise, ultimately improving the security posture of software systems. This approach could significantly enhance software security by preventing vulnerabilities from reaching production environments.

4) Future Research Opportunities: While the results are promising, several opportunities remain to further enhance the model's capabilities:

- **Enhanced Detection of Logic Flaws:** Future research should focus on improving the model's ability to detect complex logic-based vulnerabilities. This may involve incorporating additional semantic information or developing hybrid models that combine deep learning techniques with other advanced methods.
- **Expanding the Dataset:** To improve the model's generalization capabilities, expanding the dataset to include a wider range of software projects and vulnerability types is essential. This would increase the model's robustness and performance across different programming environments.
- **Model Interpretability:** Given the importance of transparency in machine learning models, particularly in security-critical applications, future work should aim to enhance the interpretability of the model's predictions. This would enable developers to better understand the rationale behind detected vulnerabilities, increasing trust in the system's recommendations.

E. Conclusion

This study demonstrates the effectiveness of deep learning-based models, particularly those utilizing Code Property Graphs (CPGs), for detecting software vulnerabilities. The model outperformed traditional static analysis techniques, providing a promising approach for identifying both common and complex vulnerabilities. However, challenges remain, particularly in the detection of logic-based vulnerabilities, which require further refinement. Future work will focus on improving the model's sensitivity to these issues, expanding the dataset, and enhancing model interpretability, all of which will contribute to making the model more applicable for real-world software security applications.

REFERENCES

[1] "Linux Linux Kernel security vulnerabilities, CVEs, versions and CVE reports," Cvedetails.com. [Online]. Available: http://www.cvedetails.com/product/47/Linux-Linux-Kernel. [Accessed: 09-Jan-2025].

[2] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *Security and Privacy (SP), 2014 IEEE Symposium on*. IEEE, 2014, pp. 590–604.

[3] N. Shervashidze, P. Schweitzer, E. J. v. Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler-lehman graph kernels," *Journal of Machine Learning Research*, vol. 12, no. Sep, pp. 2539–2561, 2011.

[4] Z. Yuan, Y. Lu, Z. Wang, and Y. Xue, "Droid-sec: deep learning in android malware detection," in *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 4. ACM, 2014, pp. 371–372.

[5] K. Kosmidis, "Machine learning and images for malware detection and classification," *IEEE Transactions on Software Engineering*, 2017.

[6] X. Liang *et al.*, "Ensemble learning for malware classification using cnns and decision trees," *IEEE Transactions on Information Forensics and Security*, vol. 15, no. 1, pp. 89–102, 2020.

[7] S. Wang, T. Liu, and L. Tan, "Automatically learning semantic features for defect prediction," in *Proceedings of the 38th International Conference on Software Engineering*. ACM, 2016, pp. 297–308.

[8] Y. Zhang *et al.*, "Vulnerability detection using graph-based deep learning models," *Proceedings of the 2019 International Conference on Security and Privacy*, pp. 234–243, 2019.

[9] M. Gori, G. Monfardini, and F. Scarselli, "A new model for learning in graph domains," in *Neural Networks, 2005. IJCNN'05. Proceedings. 2005 IEEE International Joint Conference on*, vol. 2. IEEE, 2005, pp. 729–734.

[10] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel, "Gated graph sequence neural networks," *arXiv preprint arXiv:1511.05493*, 2015.

[11] M. Niepert, M. Ahmed, and K. Kutzkov, “Learning convolutional neural networks for graphs,” in *Proceedings of the 33rd annual international conference on machine learning. ACM*, 2016.
[12] J. Lee *et al.*, “Graph neural networks for security vulnerability detection in software code,” *IEEE Transactions on Information Security*, vol. 28, pp. 322–334, 2020.
[13] X. Liu *et al.*, “Graph attention networks for vulnerability detection in software code,” *IEEE Access*, vol. 9, pp. 5628–5639, 2021.
[14] “The promise repository of empirical software engineering data,” 2015.
[15] S. Neuhaus, T. Zimmermann, C. Holler, and A. Zeller, “Predicting vulnerable software components,” in *Proceedings of the 14th ACM conference on Computer and communications security*. ACM, 2007, pp. 529–540.